

PERSPECTIVE PAPER

Of software and change

C. Ghezzi 

DeepSE Group, DEIB, Politecnico di Milano,
Milano, Italy

Correspondence

C. Ghezzi, Dipartimento di Elettronica,
Informazione e Bioingegneria-DEIB,
Politecnico di Milano, Piazza L. da Vinci, 32,
Milano, Italy.
Email: carlo.ghezzi@polimi.it

Funding information

European Commission through an Advanced
Research; Programme IDEAS ERC, Project,
Grant/Award Number: 227977-SMScom

Abstract

Change has been recognized as the distinguishing feature that makes software different from any other human-produced artifacts. Initial reflections on the urgent and unavoidable need to master change date back to the 1970s. However, despite the continuous progress that characterized software technology since, in practice, software change is still often handled as an afterthought, in an ad hoc and unprincipled manner. Agile development methods have been proposed and are now widely adopted to accommodate change during development. Recent extensions to also include operation—known as DevOps—are increasingly and successfully adopted by industry. Still, principled and rigorous foundations that can be taught, practiced, and replicated systematically are lacking.

This paper argues that change has to become a first-class concept and that the development tools used by engineers and the runtime environment supporting software execution should be structured in a way that naturally accommodates change. It also provides a perspective along which several research approaches that were investigated by the community in the past decade might be integrated and extended to make this vision become true.

Two main change categories are identified—evolution and adaptation—along with the forces that drive them. The paper discusses when and how the developed software can be designed in a way that it can self-adapt. It also discusses how the software itself can cooperate with humans in-the-loop to help them in their design and development efforts. Finally, it outlines a roadmap of future work needed to progress in the direction of supporting and automating software change that would lead to dependable adaptation and evolution.

KEYWORDS

agile processes, change, development, DevOps, formal model, incrementality, software, operation, verification

1 | INTRODUCTION

The lack of physical constraints makes software the most flexible artifact ever invented by mankind. Changing software is in principle trivial. If the source code is available, a programmer can manipulate it as text, deleting and adding new statements to produce any change. Through simple textual operations, software engineers can add, delete, or modify functionalities offered by the software and improve its qualities (eg, its performance). In more technical terms, both the functional and the nonfunctional requirements may be changed.

Despite the apparent simplicity of change making, achieving the desired goals through change is hard. Changes are made at a very low level (statement level), while the requests for change are dictated by higher level goals, such as adding new functionalities, or speeding up execution, or improving usability. Making sure that changes achieve the new goals, while preserving satisfaction of all other unchanged goals, is very often extremely hard. This is the reason why software developers are often afraid of change and make all possible efforts to avoid it.

Traditional software development processes have been structured in a phased, rigidly planned manner that ideally should eliminate the need for reconsidering and changing previous design decisions. Changes, however, are inevitable, no matter how carefully the ideal process is followed. This has been recognized since the 1970s.^{1,2} Not only changes are needed to eliminate possible defects introduced by developers but also they are at least equally needed to adapt software to changes in its operational environment, enhance its functionalities, or improve its qualities.

Traditionally, change activities to modify an existing software were collectively classified as *maintenance*, and maintenance was further decomposed into *corrective*, *adaptive*, *perfective*, and even *preventive* maintenance. Often cited folk data tell us that maintenance costs are

normally higher than development costs. Maintenance was viewed as nuisance, rather than an opportunity: a costly and largely undesirable—yet unavoidable—activity that drains resources from mainstream development activities.

As mentioned, however, change is inevitable. In most practical cases, requirements for a given application are only vaguely known when a new development starts. They become progressively better known as development proceeds, and in particular, as feedback information starts flowing from customers and from operation. Knowledge about the operational context in which the software will be embedded is often uncertain at development time. Moreover, even if the requirements and the context are known in the initial stage, they are subject to changes, which may, for example, be due to changes in the wider business context where the application is embedded. For this reason, alternative software process models have been proposed and practiced, to accommodate change into the process, to support iterative and incremental development and better align the product to evolving requirements. The term *agile software development* has become popular to collectively characterize a number of industrial efforts aimed at reducing the cost of changes in requirements by having multiple short development cycles, rather than a long monolithic one. For a presentation of the different proposals and a discussion of pros and cons, the reader can refer to Meyer.³

The way agile methods address dependability is primarily through testing. Testing has been effectively integrated into agile development lifecycles, leading to what is often called *test-driven development*. Test cases are defined before starting implementation of a new application fragment and may be viewed as a specification for the fragment. They are run on the fragment as soon as it is implemented.

Agile methods focus on adding flexibility and supporting change in the development process. They do so through a feedback loop that involves customers and leads to progressive calibration of requirements. The development cycle, however, is part of an outer cycle that also includes operation. The observations gathered on the running software may lead to further changes, which need to be designed, implemented, and then deployed and instantiated. The integration of development and operations in an overall agile framework became known as *DevOps*.⁴

In this paper, we describe an approach to ground change on formal foundations. The approach is largely based on results that were investigated by the author in the context of the project *SMScom*^{*} and reported in coauthored research articles that have been published earlier elsewhere (eg, previous studies^{5–11}). We start in Section 2 by recalling a seminal contribution by M. Jackson and P. Zave,¹² which digs into the deep causes of change and provides a formal setting for analyzing it. This work sheds the light on the impact on the software due to exogenous changes, which may lead to requirements violations. The discussion will lead to a useful classification of changes into 2 main categories: *evolution* and *adaptation*. Adaptation denotes software changes that are induced by changes in the context in which the software is situated. Recent research has studied when and how adaptation can be automated, leading to *self-adaptive software*. Evolution instead indicates software changes that are due to changes in the requirements. Evolution intrinsically requires off-line human intervention to modify the software. Likewise, human intervention is required whenever attempts to self-adapt to contextual changes fail. Section 3 provides an ideal reference framework where (self-)adaptation and evolution are fully supported. The framework can be viewed as a formal engine supporting DevOps. We then discuss how such a framework can be realized. Section 4 provides a view of how software can be equipped with self-adaptive capabilities and how these affect its run-time architecture. Section 5 discusses how human intervention can be systematically integrated and supported in the development of software and how continuous deployment can be achieved. The overall goal of this work is to show that the current informal and often unstructured practices and techniques that support DevOps may in the future be systematized, formalized, integrated, and automated. Advances in recent research have made fundamental steps in this direction (see Section 6), but we still have a long way to go to turn the vision into practice, as discussed in Section 7.

2 | A SYSTEMATIC FRAMEWORK FOR SOFTWARE DEVELOPMENT AND OPERATION

In this section, we present the conceptual framework proposed by M. Jackson and P. Zave,^{13,14} which allows us to understand requirements, software, and change. In their foundational work, Jackson and Zave observe that in requirements engineering, one needs to carefully distinguish between 2 main concerns: the *world* and the *machine*. The machine is the system of interest to be developed; the world (the environment) is the portion of the real-world affected by the machine. The ultimate purpose of the machine is always to be found in the world: The goals to be met and the *requirements* are ultimately dictated by what has to be achieved in the world and must be expressed in terms of the phenomena that occur in it. Some of these phenomena are *shared* with the machine: They are either controlled by the world and observed by the machine—through sensors—or controlled by the machine and observed by the world—through actuators. The machine is built exactly for the purpose of achieving satisfaction of the requirements in the real world. Its *specification* is a prescriptive statement of the relation on *shared phenomena* that must be enforced by the system to be developed. The implementation must be correct with respect to the specification.

The task of software engineers is to first develop a specification and then an implementation for a machine that achieves requirements satisfaction. To this end, they need upfront to understand the requirements well enough. In addition, *domain* or *environment knowledge* plays an essential role. That is, software engineers need to understand how the affected portion of the world—the embedding environment—behaves and they must formulate a set of relevant environment assumptions, which may affect satisfaction of the desired requirements. Quoting from Zave and Jackson,¹⁴

“The primary role of domain knowledge is to bridge the gap between requirements and specifications.”

^{*} Advanced Research Grant funded by European Commission: Programme IDEASERC, Project 227977-SMScom

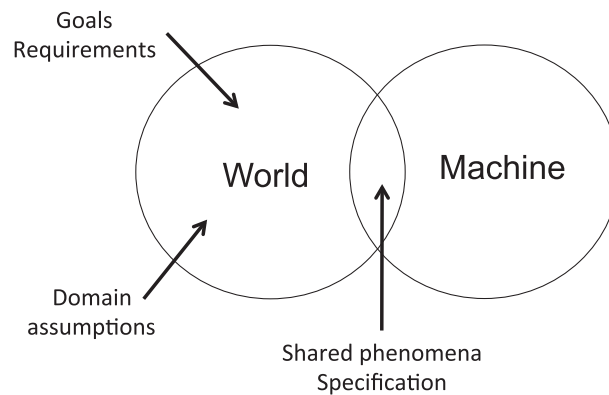


FIGURE 1 The Jackson/Zave framework

Jackson and Zave also provide a formal logical framework to describe their approach. Let R be a set of logical statements that formalize the requirements, and let S be a set of logical statements that specify the machine; let D be a set of logical statements that formalize the domain knowledge. Assuming that S and D are both satisfied and consistent with each other, the designer's responsibility is ultimately to ensure that

$$S, D \models R,$$

ie, assuming that the environment behaves according to the assumptions D , the specification S for the software under development entails satisfaction of the requirements R . We call this the *main dependability argument*.

Figure 1 provides a visual sketch of the Jackson/Zave approach. According to the previous discussion, the environment assumptions D play a fundamental role in satisfying the requirements. We need to know upfront how the environment in which the software is embedded works, since the software to develop (the machine) can achieve the expected requirements only based on the assumptions D . Should the environment behave in a way that contradicts the statements in D , the software developed according to the current specification might lead to violation of R .

Unfortunately, getting environment assumptions right is hard. The statements expressed by D may fail to capture the environment's behavior for 3 main reasons: (1) the initial domain analysis was flawed, (2) there was uncertainty about the environment when the software was initially developed, or (3) changes occurred after the software was initially developed, which invalidate the earlier assumptions. As an example, an exceptional and unexpected traffic of submitted user requests may generate an unanticipated denial of service. It is important to realize that design-time uncertainty and changes due to environmental variability are very common sources of software failures. These occur frequently, for example, in cyber-physical systems, which interact in complex ways with the physical world through sensors and actuators.

Software evolution refers to changes in the software that enable it to respond to changes in the requirements. The term *adaptation* is instead used in this work to indicate software changes that are dictated by changes in the environment, while *self-adaptation* indicates changes that can be handled autonomously by the machine. Another source of changes is originated by defects in the implementation, which may lead to failures or may lead to undesirable behaviors. In this paper, however, we do not focus on this latter class of changes.

The management of evolution in traditional software is performed off-line, during the *maintenance* phase. The traditional classification into perfective, adaptive, and corrective maintenance can also be explained by referring to the Jackson/Zave framework. Changes in the requirements, dictated by changes in the business goals of organizations or new demands by users, cause *perfective maintenance*. Environmental changes affecting domain assumptions, which may represent organizational assumptions or conditions on the physical context in which the software is embedded, cause *adaptive maintenance*. *Corrective maintenance* is instead caused by failure of the dependability argument and forces the specification to change.

In a traditional development environment, to undergo maintenance, software returns into its development stage, where changes are analyzed, prioritized, and scheduled. Changes are then handled by modifying the design and implementation of the application. The evolved system is then verified, typically via some kind of regression testing.

This paradigm does not meet the requirements of current application scenarios, which are subject to continuous changes in the requirements and in the environment, and which require rapid reaction to such changes. The off-line development cycle and the adaptive on-line cycle must proceed in parallel and suitable synchronization mechanisms must be put in place to support their coordination. This issue is further elaborated in the next section, which provides a reference ideal integrated framework.

3 | A HOLISTIC VISION

As we mentioned, modern software development must cope with continuous and rapid changes in requirements, reduced time to market, and flexible adaptation to business changes. *Agile development* has emerged as a practical effective solution to these problems. An agile approach views software development as an incremental and iterative endeavor. By following the currently widely advocated *DevOps* culture, agility extends in a seamless manner to operations: precisely, *development* and *operation* are viewed as 2 integrated concurrent and interacting perpetual processes.

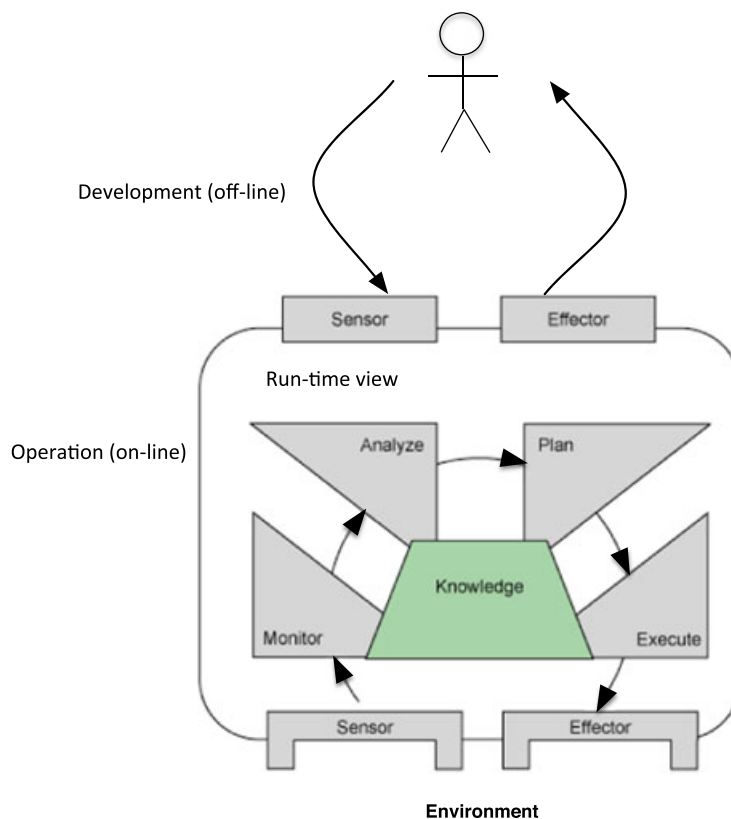


FIGURE 2 The development and operation process

Figure 2 illustrates this view of continuous development and operation, through 2 main interacting loops: the (off-line) development loop and the (on-line) self-adaptation loop performed by the software at run-time. The on-line process reifies the concept of *autonomic computing*, originally proposed by Kephart and Chess,¹⁵ which enables self-adaptation. Software engineers are in the loop and drive evolution. They get informed about the system's dynamic behavior by leveraging monitored data from the field. They are responsible for identifying new requirements and evolving the application accordingly. They are also responsible for off-line adaptation in all cases where self-adaptation fails. At any time, they can decide that newly developed or updated components should be transferred to operations to replace faulty or outdated functionalities, add new ones, or enhance existing ones. It is also possible to perform updates in a fully dynamic manner, when software is running and offering service. This is necessary in the case of systems that are required to be always up and running.

In this paper, we embrace this holistic view and discuss the role that formal methods can play to support continuous evolution and adaptation in a dependable manner, ie, where the designer focus is constantly driven by the need to formally guarantee satisfaction of the dependability argument.

The next section focuses on the running software and discusses how software can be designed to support run-time self-adaptation. Thus, during execution, the software can decide to reconfigure itself safely to respond to changes in the environment.

4 | TOWARDS SELF-ADAPTATIVE SOFTWARE: OPERATION

The goal of making software self-adaptable has become a hot research topic in the last decade, and many promising results have been proposed. For a broad view of the area, the reader may refer to the series of SEAMS workshops and symposia[†] and the 2 Dagstuhl reports.^{16,17}

Engineering self-adaptive systems calls for specific new approaches to the development and operation of software that guarantee lifelong requirements fulfillment in the presence of environmental changes. Hereafter, we focus on self-adaptation in the context of *nonfunctional requirements*, such as reliability, performance, and different kinds of cost-related requirements, such as energy consumption. Nonfunctional requirements are often very sensitive to changes in the environment. We describe an approach that can (1) predict possible requirements failures caused by changes occurring in the environment and (2) self-adapt by triggering appropriate countermeasures that dynamically reconfigure the running application. In the sequel, we use the term *failure* to indicate any violation of (nonfunctional) requirements. For a further discussion about the definition of nonfunctional requirements and failures, the reader can refer to Ghezzi and Tamburrelli.⁶

[†]<http://www.self-adaptive.org>

To be self-adaptive, a software system must be able to (1) detect the relevant changes in the external world in which it operates, (2) reason about its own ability to continue to fulfill the requirements as a consequence of the detected changes, and (3) reconfigure itself to guarantee a seamless adaptation to the new external conditions.

Several promising approaches to software self-adaptation rely on the use of *models at run-time*.^{8,18} Our work fully embraces this view. We keep models at run-time and update them automatically as changes are dynamically discovered through monitoring (see previous studies^{5,19-22}). We advocate the use of formal models that support reasoning about possible requirements violations. More specifically, we have been studying the use of different kinds of operational models, whose state space can be systematically and exhaustively explored through *on-line model checking* against a formal description of requirements expressed as logic statements. The outcome of model checking may trigger proper adaptation strategies to steer system reconfigurations and prevent requirements violations. Conceptually, this framework establishes a *feedback control loop* between models and the running system. At run-time, monitored environment data are fed back to generate possible model updates, which are in turn analyzed against requirements. Adaptation thus becomes model-driven. This approach reflects the *autonomic control loop* advocated in Kephart and Chess¹⁵ and illustrated (in slightly different terms) in Figure 3.

Since we restrict our focus to nonfunctional properties, we discuss here the use of *Markov models*, which nicely support quantitative probabilistic specifications as well as formalization and verification of requirements, such reliability, performance, and costs, through a *probabilistic temporal logic* and *probabilistic model checking*. Model checking techniques rely on well-known and effective algorithms that are available as powerful, off-the-shelf, open-source tools, such as PRISM²³ and MRMC.²⁴

As an example, consider the case shown in Figure 4, originally introduced in Ghezzi and Tamburrelli²⁵ and then also used in the tutorial paper.¹⁰ The figure shows an abstract view of a software that is built as a workflow that integrates a number of third-party services offered by other organizations. The workflow, in turn, interacts with users, supporting their interactive transactions. In particular, let us consider a particular case of the generic system shown in Figure 4, which supports an interactive e-commerce application that sells merchandise on-line to end users. In this case, the third-party services integrated by the workflow are

1. *Authentication Service*, supporting *Login* and *Logout* operations through which the system authenticates users.
2. *Payment Service*, providing safe transactional payments (*CheckOut* operation).
3. *Shipping Services*, supporting purchase delivery. Two kinds of shipping services are provided: *NrmShipping* and *ExpShipping*, for faster but more expensive delivery.

Logged users are classified as *NewCustomer* or *ReturningCustomer*, based on their usage profile. The idea is that different quality of service should be offered to users of different classes.

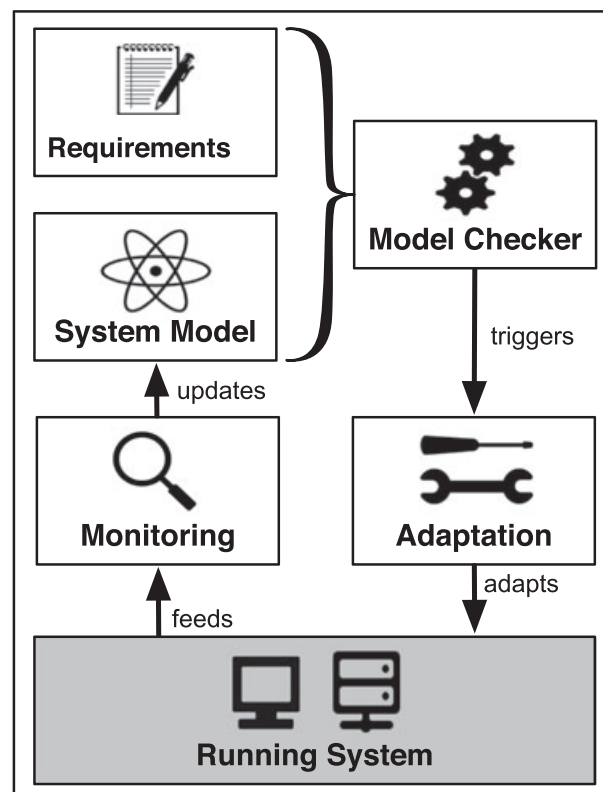


FIGURE 3 Model-checking at run-time

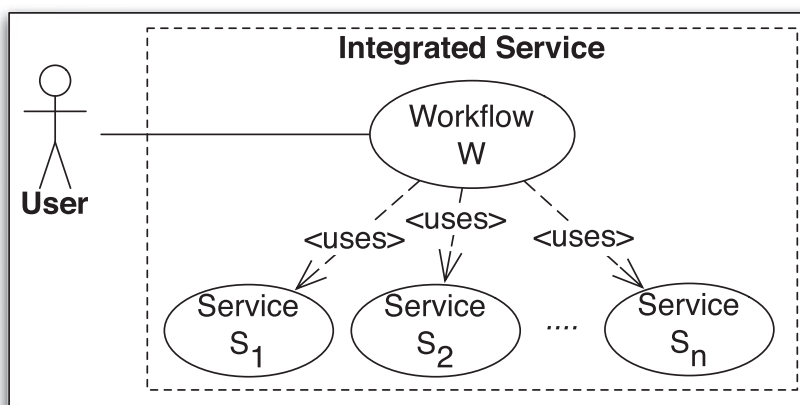


FIGURE 4 A class of applications

TABLE 1 Domain assumptions on usage profiles

Description	Value
$P(\text{User is a PC})$	.35
$P(\text{PC chooses express shipping})$	.5
$P(\text{NC chooses express shipping})$	.25
$P(\text{PC searches again after a buy operation})$	.2
$P(\text{NC searches again after a buy operation})$	.15

Abbreviation: NC, NewCustomer.

TABLE 2 Domain assumptions on external services

Description	Value
$P(\text{Login})$	.03
$P(\text{Logout})$	.03
$P(\text{NrmShipping})$	.05
$P(\text{ExpShipping})$	.05
$P(\text{CheckOut})$	.1

The system must satisfy certain QoS requirements, such as a reliability requirement for purchase transactions. This may be formulated as the probability that customers successfully terminate interactive sessions where they browse through the catalogue of offers, select purchases, put them in a shopping chart and then proceed to check out and payment. The requirement can be stated as “*The probability of success of interactive sessions must be greater than 0.8.*” To design a workflow that satisfies all requirements, software engineers must make assumptions about the behavior of 2 main components of the environment: users and third-party services. Assumptions about user behaviors describe the expected usage profile. A possible example is shown in Table 1, where the notation $P(x)$ indicates the probability of “ x ”. Table 2, instead, summarizes the assumptions about the behavior of the external services, where $P(Op)$ here denotes the probability of failure of service operation Op . These assumptions derive, most likely, from the service-level agreement with the providers of third-party services integrated within the workflow under design.

An initial model of the e-commerce application is shown in Figure 5 as an activity diagram. From this model, and taking into account the environment assumptions represented in Tables 1 and 2, the designer produces an enriched state-machine model—a *Discrete-time Markov chain (DTMC)*—that formally captures the behavior of both the application under development (the *machine*) and the environment. Transitions describe interactive operations, according to the protocol specified by the activity diagram of Figure 5. Domain assumptions are modeled as probabilities that label the transitions. The model also represents failure and success states.

The high-level design represented in Figure 6 can be formally validated against the requirements. This can be done by encoding the requirements as probabilistic temporal logic formulae (in the example stated as *the probability of eventually reaching state 16*) and then running a model checker on the DTMC in Figure 6. In our example, it can be shown that the aforementioned reliability requirement is satisfied by the high-level design.

We discuss now what can happen at run-time, when the e-commerce application is running and providing service to customers. The assumptions made at design-time about the environment in which the e-commerce system is embedded may be invalidated for different reasons. The failure may occur as a consequence of an upfront wrong understanding of the environment, or because it has changed its behavior since. Consider, for example, user profiles, where the ratio between occasional and returning customers may change. Consider also reliability of integrated services, which may change due to unexpected changes in their implementation performed by service providers. If the application's model is kept alive at run-time, one

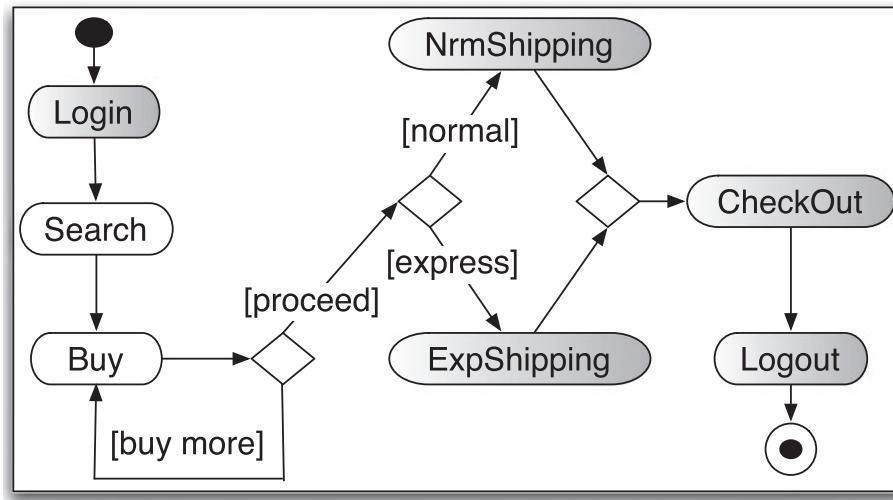


FIGURE 5 Operational description of the specification via an activity diagram

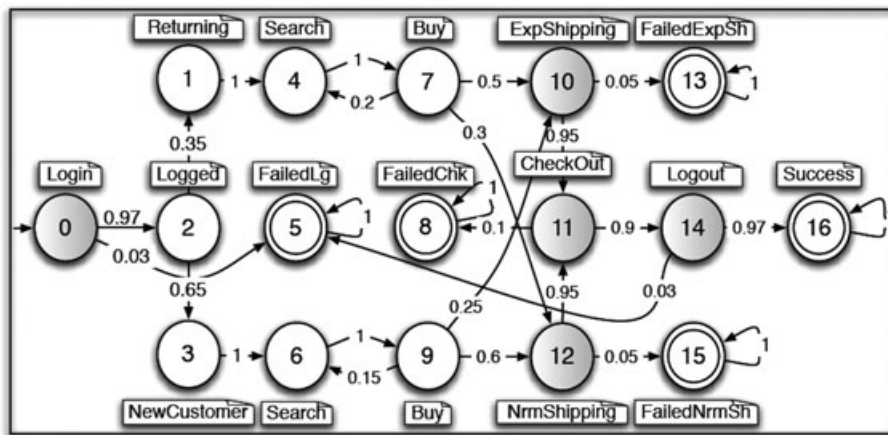


FIGURE 6 Discrete-time Markov chain model for the case study

can monitor the environment for possible changes, modify the model accordingly (ie, modify the values of probabilities of certain transitions based on monitored data), and check whether verification fails or not. If it does, the application may try to self-adapt—eg, it may try to rebind to different external services.

This approach, however, does not scale as desirable to practical systems. Model-checking techniques and tools, in fact, were defined originally to support off-line development-time analysis and are not meant for on-line usage, where they would need to comply with strict real-time constraints. Run-time verification must, in fact, support timely adaptation, to avoid unacceptable disruption in service provision. Considerable research efforts have been focusing exactly on how to bring model checking to run-time.

A possible solution to this problem is based on the consideration that changes often are not radical and have a local scope. It is possible to take advantage from this assumption by making model checking *incremental*. A way to do that has been presented in Filieri et al²⁰ for models described as DTMCs and for temporal probabilistic requirements. It is based on the assumption that when the system is designed, engineers know which model parameters (probabilities associated with transitions) represent changeable assumptions. Given a set of requirements, encoded as probabilistic temporal formulae, the approach statically generates a set of expressions encoding the verification conditions to be evaluated at run-time to verify satisfaction of system requirements. Run-time verification becomes very efficient, since it simply requires evaluating verification conditions instead of running a model checker. Run-time efficiency is of course achieved at the expense of a more complex model checking process that takes place at design-time, which requires mixed symbolic and numeric computations (see Filieri et al⁷ for details.)

To conclude, this discussion has shown that self-adaptation to changes in the environment can be achieved under the following assumptions:

1. During design, it is possible to identify the possible sources of change.
2. A parametric model can be produced for the system where parameters encode environmental conditions that may change and become known during operation.
3. Verification conditions can be generated for the parametric model and for all formally specified requirements.

4. Environmental data can be monitored at run-time to identify the actual values of model parameters to be fed into the verification conditions.
5. The application is structured in a way that it can be reconfigured dynamically to accommodate run-time parameter variability.

Regarding the last point, different techniques may be adopted. In the example discussed above, it is possible to achieve that through dynamic binding to external services. It is also possible to address the problem by designing the application as a dynamic software product line.²⁶ In any case, dynamic reconfiguration must occur while the application is running. Several techniques have been devised to support dynamic software reconfigurations in a completely safe, nondisruptive, and efficient way.²⁷⁻²⁹

Finally, one should take into account the possibility that the implemented self-adaptation policy fails: The system cannot self-adapt to the detected changes occurring in the environment. In this case, it is necessary to resort to human intervention to drive software evolution. The collected data that provide evidence of failure are valuable knowledge that can be presented to off-line engineers to support evolution.

5 | SELF-ADAPTIVE SOFTWARE: DEVELOPMENT AND THE INTERPLAY WITH OPERATION

Agile software development mandates progressing in an iterative and incremental manner to support continuous change. In the initial phases of a new project, this is caused by the need to calibrate requirements and explore different design alternatives. Subsequently, as soon as a version of software is deployed and running, development continues in parallel to produce new versions, which may, for example, improve satisfaction of nonfunctional requirements, meet additional requirements, or deal with environment changes that were not anticipated and not supported by self-adapting policies of the current operational versions.

Current state-of-the-art practices in agile development mainly address the organizational aspects involved in iterative and incremental development. To support product quality assurance, they advocate *test-driven development*. In essence, it prescribes that each product increment should be initially specified by designing the test cases the implementation should eventually pass and then relies on automation of test case execution. Although this emphasis on continuous quality assurance is commendable, we strongly argue that the current state of practice is still far from being satisfactory. More can and should be done to improve and assure quality, and hence dependability of software products. In particular, advances in formal methods (formal modeling and formal verification) have lead to results that can be incorporated in development environments to enrich the workbench available to software engineers. This does not come for free though, but rather calls for software engineering research to align formal methods to the needs of practitioners. Hereafter, we mention 2 important areas in which research progress has indicated that time is now mature for serious investments to lead to advanced tool development and practical usage.

Recent research has been focusing on partial, incomplete formal models, and techniques to formally verify them in face of their evolution. This is extremely important to support agile modeling.^{9,30} Evolution and incompleteness go hand in hand: They are 2 aspects of the same problem. An incomplete specification evolves into a complete one once the unknown or uncertain aspects become known. By following the principle of separation of concerns, parts of the system can be deliberately left incomplete at a given stage, and their completion is postponed to a later stage. Another kind of evolution regards the support to exploration of alternative designs. This may be viewed as consisting of 2 steps: First, the model is put in an incomplete state by deleting parts that are then completed in a second step. By verifying a model vis-a-vis its evolution, designers may assess the impact of changes.

In an agile setting, as discussed, software development is structured through frequent iterations; hence, iterations must be supported efficiently. Hereafter, we focus on verifying iterations through *model checking*. Model checking can be applied at different stages of software development. It can be applied to models (typically, operational models based on state-machine descriptions) and also to code.

To fit an iterative development, the model-checking procedure should support both *reusability* and *incrementality*.³¹ Reusability matters because changes to a model may have a local impact, so redoing the whole verification after any change would be very inefficient. Let us assume, for example, that a large-scale workflow successfully satisfies a certain property; a new activity is later added to the workflow, requested by customers. By following most existing monolithic model-checking approaches, the new versions of the workflow would need to undergo a complete recheck against the property. If changes are frequent, formal verification becomes a bottleneck of the development process. Furthermore, as we mentioned, in the real-world, systems are often designed through the iterative decomposition, to support prioritization of different parts and separate developments. This approach requires that it should be possible to complete a specification incrementally, as the different parts are designed and developed. It would thus be useful to be able to check if an incomplete specification meets the specified requirements. In the likely case that satisfaction of the global property depends on the missing components, it would be desirable to know under which constraints the missing parts should be designed so that the global property holds for the complete specification. Satisfaction of these constraints is later performed by analyzing only the added completion. The benefit of such incomplete model checking is that the unavailable components can be independently developed and verified regardless of the rest of the system.

We explored how to support an incremental approach in,³² where we describe a new model-checking procedure for finite-state machines where certain states can be labeled as *black-box* states, indicating that they encapsulate an unspecified behavior, whose design is deferred to a later stage. The verification procedure checks the incomplete design against a formal property in temporal logic, expressing a global system requirement. The outcome of verification can be *OK*, if the model satisfies the property; *KO* if it does not, independent of incompleteness. The outcome can also be

MAYBE, if there is a possible refinement of the black-box that may lead to a violation. In this last case, the verification procedure also synthesizes a formal property (called *proof obligation*) that expresses a constraint on the unspecified part to be met to satisfy the global requirement.

Another relevant line of research has focused on making analysis incremental. Incrementality is a necessary feature to be supported if one wishes to make formal verification practically usable. Since iterative development is based on continuous relatively small changes, the ability to verify if changes keep satisfying requirements is of paramount importance. An incremental verification approach reuses the results of previous analysis to verify an artifact after change, and tries to minimize the portion of new analysis to be performed. It has benefits over a nonincremental approach in terms of speed. It also has benefits since it helps focusing on the scope of a change. The principle of incrementality can be applied to verification of any artifact, eg, both models³³ and code.³⁴

6 | RELATED WORK

Research on self-adaptive software has been quite active since more than a decade. The approach presented in this paper to support self-adaptation via model-and-verification at run-time and the work discussed in Sections 4 and 5 have been developed by the author in the context of the project *SMScom*.[‡] For a general understanding of the main approaches investigated in this area, the reader is directed to the proceedings of the SEAMS symposium[§] and the books produced by a series of Dagstuhl seminars.^{16,17}

To support modeling of incomplete systems, different formalisms have been proposed, such as Modal Transition Systems,³⁵ Partial Kripke Structures,³⁶ λ KSS³⁷, and LTS[†].³⁸ Other formalisms, such as Hierarchical State Machines,³⁹ are used to model sequential processes via a top-down development process.

Incremental verification is the subject of many research efforts. The most common approach equates the notion of increment with the notion of module. It subdivides the system to be verified into different modules and verifies them separately via the assume-guarantee approach⁴⁰; this approach is used, for example, in Cobleigh et al⁴¹ and Kwiatkowska et al.⁴² This concept has been extended with the definition of the substitutability problem,^{43,44} which consists in verifying that a module can be safely replaced by another.

Henzinger et al⁴⁵ achieve incrementality by the computation of the difference in the abstract reachability tree, which determines the parts that need to be analyzed. Instead, Lauterburg et al⁴⁶ provide an incremental explicit space-state model-checking procedure. It performs a graph search from the initial state, trying to reach states that violates the given properties avoiding to visit the same state many times. The incrementality is obtained by analyzing the difference in the state-space graph to find what explorations have to be performed. A similar work is described by Yang et al,⁴⁷ which can perform the new analysis skipping the parts of the state-space, which will not change behaviour after the changes. In Conway et al⁴⁸ the analysis is recorded in a "derivation graph". After a change, it is altered by adding the new derivations and removing the ones no more present in the program. Person et al⁴⁹ apply a similar approach to symbolic execution. It performs a control flow graph analysis recomputing only the affected paths. All these techniques have the disadvantages of being bound to the model used to represent the state-space and thus are difficult to adapt for a different one.

We are currently working on developing a general approach to incremental verification, which can be instantiated to different kinds of models and notations.¹¹

7 | FUTURE WORK AND CONCLUSIONS

Change is an intrinsic feature of software. It makes software special and different from other artifacts produced by humans. Despite the tremendous progress in software science and technology and the amazing practical achievements, the way change is managed is still rather primitive. We suffer from it rather than take advantage; it is viewed as an afterthought rather than as a foundational principle. Although the increasing success of agile approaches indicates that the situation is changing, agile approaches are still practiced in an informal and largely unprincipled manner. They lack the systematic, rigorous, formal foundations that are needed to conjugate change with assurances, as required by the increasing pervasiveness of software technology in all sectors of society. In this paper, we claim that time is now mature to integrate formal modeling and verification into agile methods to improve quality and achieve higher assurance levels.

Besides the work cited in the previous section, advances in research on model-driven development,⁵⁰ self-adaptive systems,^{16,17} run-time verification,^{8,51-53} dynamic software updates,²⁷⁻²⁹ to name a few relevant areas, provide numerous important background results that can pave the way for future research that should address how to engineer integrated environments and methodologies that fully support agile development and operations through the discipline and rigor of formal methods, in a usable and nonintrusive way. This will require a collective effort by the software engineering research community, collaboration with other nearby communities, and interaction with software engineering practitioners.

[‡] Advanced Research Grant funded by European Commission: Programme IDEASERC, Project 227977-SMScom.

[§] <http://www.self-adaptive.org>

ACKNOWLEDGEMENTS

The work described in this paper was funded by the European Commission through an Advanced Research Grant, Programme IDEAS ERC, Project227977-SMScom.

ORCID

C. Ghezzi  <http://orcid.org/0000-0002-7234-5011>

REFERENCES

1. Belady LA, Lehman MM. A model of large program development. *IBM Syst J*. 1976;15(3):225-252.
2. Lehman MM, Belady LA, eds. *Program Evolution: Processes of Software Change*. Academic Press Professional, Inc.; 1985.
3. Meyer B. *Agile!: The Good, the Hype and the Ugly*. Springer Science & Business Media; 2014.
4. Bass L, Weber I, Zhu L. *Devops: A Software Architect's Perspective*. Addison-Wesley Professional; 2015.
5. Epifani I, Ghezzi C, Mirandola R, Tamburrelli G. Model evolution by run-time parameter adaptation. In: *Software Engineering, 2009. ICSE 2009. IEEE 31st International Conference on IEEE*; 2009:111-121.
6. Ghezzi C, Tamburrelli G. Reasoning on non-functional requirements for integrated services. In: *17th IEEE International Requirements Engineering Conference, RE'09. IEEE*; 2009:69-78.
7. Filieri A, Tamburrelli G, Ghezzi C. Supporting self-adaptation via quantitative verification and sensitivity analysis at run time. *IEEE Trans Software Eng*. 2016;42(1):75-99.
8. Calinescu R, Ghezzi C, Kwiatkowska M, Mirandola R. Self-adaptive software needs quantitative verification at runtime. *Commun ACM*. 2012;55(9):69-77.
9. Ghezzi C, Sharifloo AM, Menghi C. *Towards Agile Verification*. Berlin, Heidelberg: Springer Berlin Heidelberg; 2013:31-47.
10. Ghezzi C. *Dependability of Adaptable and Evolvable Distributed Systems*. Cham: Springer International Publishing; 2016:36-60.
11. Bianculli D, Filieri A, Ghezzi C, Mandrioli D. Syntactic-semantic incrementality for agile verification. *Sci Comput Program*. 2015;97:47-54.
12. Zave P, Jackson M. Four dark corners of requirements engineering. *ACM Trans Software Eng Methodology (TOSEM)*. 1997;6(1):1-30.
13. Jackson M, Zave P. Deriving specifications from requirements: an example. In: *ICSE '95: Proceedings of the 17th International Conference on Software Engineering*. New York, NY, USA: ACM; 1995:15-24.
14. Zave P, Jackson M. Four dark corners of requirements engineering. *ACM Trans Softw Eng Methodol*. 1997;6(1):1-30.
15. Kephart JO, Chess DM. The vision of autonomic computing. *IEEE Comput*. 2003;36(1):41-50.
16. Cheng BHC, De Lemos R, Giese H, et al. Software engineering for self-adaptive systems: A research roadmap. In: *Software Engineering for Self-Adaptive Systems*. Springer; 2009:1-26.
17. De Lemos R, Giese H, Müller HA, et al. Software engineering for self-adaptive systems: A second research roadmap. In: *Software Engineering for Self-Adaptive Systems II*. Springer; 2013:1-32.
18. Blair G, Bencomo N, France RB. Models@ run. time. *Comput*. 2009;42(10):22-27.
19. Ghezzi C, Tamburrelli G. Predicting performance properties for open systems with kami. In: *Architectures for Adaptive Software Systems*. Springer; 2009:70-85.
20. Filieri A, Ghezzi C, Tamburrelli G. Run-time efficient probabilistic model checking. In: *Proceedings of the 33rd International Conference on Software Engineering, ICSE '11*. New York, NY, USA: ACM; 2011.
21. Filieri A, Ghezzi C, Tamburrelli G. A formal approach to adaptive software: continuous assurance of non-functional requirements. *Formal Aspects Comput*. 2012;24(2):163-186.
22. Ghezzi C, Pinto LS, Spoletini P, Tamburrelli G. Managing non-functional uncertainty via model-driven adaptivity. In: *Proceedings of the 2013 International Conference on Software Engineering*. IEEE Press; 2013:33-42.
23. Hinton A, Kwiatkowska M, Norman G, Parker D. Prism: A tool for automatic verification of probabilistic systems. *TACAS*. 2006;3920:441-444.
24. Katoen J-P, Khattri M, Zapreev IS. A Markov reward model checker. In: *QEST*. Los Alamos, CA, USA: IEEE Computer Society; 2005:243-244.
25. Ghezzi C, Tamburrelli G. Reasoning on non-functional requirements for integrated services. In: *Proceedings of the 17th International Requirements Engineering Conference*. IEEE Computer Society; 2009:69-78.
26. Hallsteinsen S, Hinchey M, Park S, Schmid K. Dynamic software product lines. *Computer*. 2008;41(4).
27. Kramer J, Magee J. The evolving philosophers problem: Dynamic change management. *IEEE Trans Software Eng*. 1990;16(11):1293-1306.
28. Vandewoude Y, Ebraert P, Berbers Y, D'Hondt T. Tranquility: A low disruptive alternative to quiescence for ensuring safe dynamic updates. *IEEE Trans Software Eng*. 2007;33(12):856-868.
29. Baresi L, Ghezzi C, Ma X, Panzica La Manna V. Efficient dynamic updates of distributed components through version consistency. *IEEE Trans Software Eng*. 2017. to appear.
30. Uchitel S, Alrajeh D, Ben-David S, et al. Supporting incremental behaviour model elaboration. *Comput Sci - Res Development*. 2013;28(4):279-293.
31. Menghi C. Verifying incomplete and evolving specifications. In: *Companion Proceedings of the 36th International Conference on Software Engineering, ICSE Companion 2014*. New York, NY, USA: ACM; 2014:670-673.
32. Menghi Claudio, Spoletini Paola, Ghezzi Carlo. Dealing with incompleteness in automata-based model checking. In: *FM 2016: Formal Methods - 21st International Symposium, Limassol, Cyprus, November 9-11, 2016, Proceedings*; 2016:531-550.
33. Bianculli D, Filieri A, Ghezzi C, Mandrioli D. Incremental syntactic-semantic reliability analysis of evolving structured workflows. In: *International Symposium on Leveraging Applications of Formal Methods, Verification and Validation*. Springer; 2014:41-55.

34. Bianculli D, Filieri A, Ghezzi C, Mandrioli D, Rizzi AM. Syntax-driven program verification of matching logic properties. In: Proc. of FME Workshop on Formal Methods in Software Engineering. IEEE; 2015.
35. Larsen KG, Thomsen B. A modal process logic. In: Proc. of LICS'88. IEEE; 1988:203-210.
36. Bruns G, Godefroid P. Model checking partial state spaces with 3-valued temporal logics. In: Proc. of CAV'99, LNCS, vol. 1633; 1999:274-287.
37. Chechik M, Devereux B, Easterbrook S, Gurfinkel A. Multi-valued symbolic model-checking. *ACM Trans Software Eng Methodology (TOSEM)*. 2003;12(4):371-408.
38. Giannakopoulou D, Pasareanu CS, Barringer H. Assumption generation for software component verification. In: Proc. of ASE'02 IEEE; 2002:3-12.
39. Alur R, Yannakakis M. Model checking of hierarchical state machines. *ACM Trans Program Lang Syst (TOPLAS)*. 2001;23(3):273-303.
40. Jones CB. Tentative steps toward a development method for interfering programs. *ACM Trans Program Lang Syst*. 1983.
41. Cobleigh JM, Giannakopoulou D, Păsăreanu CS. Learning assumptions for compositional verification. In: Proc. of TACAS'03. Springer; 2003:331-346.
42. Kwiatkowska M, Norman G, Parker D, Qu H. Assume-guarantee verification for probabilistic systems. In: Proc. of TACAS'10, LNCS, vol. 6105. Springer; 2010:23-37.
43. Chaki S, Sharygina N, Sinha N. Verification of evolving software. 2004.
44. Sharygina N, Chaki S, Clarke E, Sinha N. Dynamic component substitutability analysis. In: Proc. of FM'05, LNCS, vol. 3582. Springer; 2005:512-528.
45. Henzinger TA, Jhala R, Majumdar R, Sanvido MAA. Extreme model checking. *Verification: Theory and Practice*. 2004:180-181.
46. Lauterburg S, Sobeih A, Marinov D, Viswanathan M. Incremental state-space exploration for programs with dynamically allocated data. In: Proc. of ICSE'08, ICSE '08. ACM; 2008:291-300.
47. Yang G, Dwyer MB, Rothermel G. Regression model checking. In: Proc. of ICSM'09. IEEE; 2009:115-124.
48. Conway CL, Namjoshi KS, Dams D, Edwards SA. Incremental algorithms for inter-procedural analysis of safety properties. In: Proc. of CAV, LNCS, vol. 3576. Springer; 2005:449-461.
49. Person S, Yang G, Rungta N, Khurshid S. Directed incremental symbolic execution. In: Proc. of PLDI'11; 2011:504-515.
50. Combemale B, France R, Jézéquel J-M, Rumpe B, Steel J, Vojtisek D. *Engineering modeling languages: Turning domain knowledge into tools*. CRC Press; 2016.
51. Bauer A, Leucker M, Schallhart C. Runtime verification for ltl and tltl. *ACM Trans Softw Eng Methodol*. 2011;20(4):14:1-14:64.
52. Colin S, Mariani L. Run-time verification. In: *Model-Based Testing of Reactive Systems*. Springer; 2005:525-555.
53. Filieri A, Ghezzi C, Tamburrelli G. Run-time efficient probabilistic model checking. In: *Proceedings of the 33rd International Conference on Software Engineering*; 2011.

How to cite this article: Ghezzi C. Of software and change. *J Softw Evol Proc*. 2017;29:e1888. <https://doi.org/10.1002/smr.1888>